

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Debugging and Optimisation task
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	RAGAVI S	Reg. No.:	192565027
Section / Batch:		Date:	05-08-2026

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

QUESTIONS

1. A computer system performing signed binary subtraction using 2's complement produces incorrect results for certain operand combinations. Debug the subtraction process by examining binary conversion, complement generation, and overflow detection. Suggest appropriate corrections to ensure accurate signed arithmetic.
2. A digital processor implementing a carry look-ahead adder fails to produce the correct sum for some input combinations. Debug the generate and propagate logic to identify the source of the error. Recommend modifications that restore correct operation while maintaining high-speed performance.
3. A hardware multiplier using Booth's algorithm consumes more clock cycles than expected during signed multiplication. Debug the execution sequence by analyzing bit-pair decisions, arithmetic shifts, and register updates. Propose optimization techniques to improve execution efficiency without affecting accuracy.
4. A processor implementing the non-restoring division algorithm produces incorrect quotient values for specific dividend and divisor combinations. Debug the step-by-step division process by examining partial remainders, shifting operations, and quotient-bit generation. Suggest corrections to obtain accurate division results.
5. A graphics processing application experiences performance degradation due to frequent floating-point computations using IEEE 754 double-precision representation. Debug the arithmetic operations to identify unnecessary precision or computational overhead. Recommend suitable optimization techniques to improve execution speed while maintaining the required numerical accuracy.

Code of Conduct

*I **RAGAVIS / 192565027** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: RAGAVI S

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Identification of Errors / Bugs	20	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

Question 1: Debugging Signed Binary Subtraction Using 2's Complement

1. Identification of Errors / Bugs

On tracing the subtraction routine $A - B$, implemented as $A + (2\text{'s complement of } B)$, the following faults are typically responsible for incorrect results:

- Incorrect complement generation: the routine sometimes takes only the 1's complement (bit inversion) of B and forgets to add 1, which silently converts the operation into 1's complement arithmetic and produces an answer that is off by one.
- Fixed word-length truncation: when B is complemented, the carry-out of the most significant bit (MSB) is discarded without being checked, so a valid carry that should indicate a sign condition is lost.
- Sign-bit misinterpretation: the result's MSB is used directly as a magnitude bit instead of being interpreted as a sign bit, which corrupts results whenever the outcome is negative.
- Missing overflow detection: the implementation does not compare the carry into the MSB (C_{in}) with the carry out of the MSB (C_{out}); overflow occurs precisely when these two differ, and omitting this check allows silently wrapped-around (incorrect) results to be reported as valid.

2. Debugging Approach and Analysis

The debugging approach traces the subtraction $A - B$ step by step for representative operand pairs, including same-sign and opposite-sign combinations, and edge cases such as the most negative representable number:

- Step 1 – Convert B to true 2's complement: invert every bit of B and add 1 (not merely invert the bits).
- Step 2 – Add A to the 2's complement of B using standard binary addition, retaining the carry out of the MSB.
- Step 3 – Discard any carry beyond the fixed word length (n bits) — this carry is expected in 2's complement arithmetic and is not itself an error.

- Step 4 – Compute overflow as $V = C_{in} \text{ XOR } C_{out}$ for the sign-bit position, and flag the result as invalid whenever $V = 1$.

Tracing a case such as $A = -8$ and $B = +1$ in a 4-bit representation shows the fault clearly: $A - B$ should equal -9 , which cannot be represented in 4 bits. Without the C_{in}/C_{out} overflow check, the hardware silently returns $+7$, an incorrect wrapped-around value that is easily mistaken for a valid result.

3. Corrected Solution

The corrected subtraction algorithm is as follows:

- Represent both operands in true n-bit 2's complement form.
- Form the 2's complement of the subtrahend (B) by inverting all bits and adding 1 to the LSB.
- Add A to the complemented B using a standard n-bit adder.
- Discard the carry-out beyond bit $(n - 1)$; it does not indicate an error in 2's complement subtraction.
- Compute overflow $V = C_{in} \text{ XOR } C_{out}$ at the sign bit; if $V = 1$, the result is invalid for the given word length and must be flagged (e.g., raise an overflow exception or widen the word length).
- Interpret the final MSB strictly as the sign bit of the corrected result, not as a magnitude bit.

4. Optimization

To improve performance without sacrificing accuracy, the complement-and-add sequence is optimized by generating the 2's complement and performing the addition in a single pass using a full adder whose initial carry-in is forced to 1 while all bits of B are fed through inverters. This removes the separate 'add 1' step, reduces one arithmetic cycle, and keeps the overflow-detection logic (XOR of C_{in} and C_{out}) integrated directly into the adder's carry chain so no additional hardware pass is required.

5. Testing and Validation

Test Case	A	B	Expected (A – B)	Result After Fix	Overflow Flag
Same sign	0101 (5)	0011 (3)	0010 (2)	0010 (2)	0 (No overflow)
Opposite sign	0111 (7)	1110 (–2)	1001 (invalid in 4-bit, correctly flagged)	Flagged	1 (Overflow)
Boundary case	1000 (–8)	0001 (1)	Not representable in 4-bit	Flagged	1 (Overflow)
Zero result	0100 (4)	0100 (4)	0000 (0)	0000 (0)	0 (No overflow)

The corrected algorithm was validated against same-sign, opposite-sign, zero-result, and boundary (most-negative-number) test vectors. All in-range results matched expected values, and all out-of-range cases were correctly flagged by the $C_{in} \text{ XOR } C_{out}$ overflow check, confirming both correctness and reliable overflow reporting.

Question 2: Debugging a Carry Look-Ahead Adder (CLA)

1. Identification of Errors / Bugs

Re-deriving the generate and propagate equations for each bit i ($G_i = A_i \cdot B_i$, $P_i = A_i \text{ XOR } B_i$) and the carry recurrence ($C_{i+1} = G_i + P_i \cdot C_i$) against the faulty design reveals the following bugs:

- Incorrect propagate function: the implementation uses $P_i = A_i + B_i$ (logical OR) instead of $P_i = A_i \text{ XOR } B_i$. OR-based propagate is only functionally equivalent to XOR when G_i is also correctly ANDed in, but in this design it causes an extra carry to be generated when $A_i = B_i = 1$, double-counting the carry already produced by G_i .
- Carry recurrence not fully expanded: the higher-order carries (e.g., C_2 , C_3 , C_4) are computed using the ripple relation $C_{i+1} = G_i + P_i \cdot C_i$ sequentially, rather than being expanded fully in terms of the primary inputs, which reintroduces the ripple delay the CLA is meant to remove and can create timing races that appear as functional errors.

- Group carry-out omitted: block/group generate and propagate terms are not computed, so cascading multiple 4-bit CLA blocks produces an incorrect carry into the next block.

2. Debugging Approach and Analysis

The debugging process compares the truth table of the faulty propagate logic against the correct XOR-based definition for all input combinations of A_i and B_i , isolating the case $A_i = B_i = 1$ as the point of divergence — this is exactly where OR and XOR disagree. The carry expressions are then re-derived symbolically:

- $C_1 = G_0 + P_0 \cdot C_0$
- $C_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$
- $C_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$
- $C_4 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$

Comparing these fully expanded (non-recursive) expressions against the faulty sequential implementation confirms that the design was effectively behaving as a ripple-carry adder with look-ahead labels, explaining both the incorrect sums for $A_i = B_i = 1$ cases and the loss of speed advantage.

3. Corrected Solution

- Redefine the propagate signal as $P_i = A_i \text{ XOR } B_i$ (exclusive-OR), matching the true CLA propagate condition.
- Keep the generate signal as $G_i = A_i \cdot B_i$.
- Implement each carry C_{i+1} using the fully expanded sum-of-products expression directly from the primary inputs, rather than chaining C_i from the previous stage, so every carry bit is computed in parallel.
- Introduce block-level generate (G_{block}) and propagate (P_{block}) terms when cascading multiple CLA blocks, so inter-block carries are also computed by look-ahead rather than by rippling.

4. Optimization

Performance is further optimized by grouping the adder into 4-bit CLA blocks with a second level of look-ahead logic between blocks (a hierarchical or 'carry look-ahead

generator' structure). This bounds the critical path to a small, fixed number of gate delays regardless of operand width, giving near-constant-time carry propagation for wide adders (e.g., 16-bit or 32-bit) while keeping the gate count manageable compared to a fully flattened single-level look-ahead design.

5. Testing and Validation

A (4-bit)	B (4-bit)	Cin	Expected Sum	Result After Fix
0110	0011	0	1001	1001 (Correct)
1111	0001	0	10000 (Cout=1, Sum=0000)	Cout=1, Sum=0000 (Correct)
1010	1010	0	10100 (Cout=1, Sum=0100)	Cout=1, Sum=0100 (Correct)
0101	0101	1	1011	1011 (Correct)

Re-simulating the corrected propagate/generate logic and fully expanded carry equations against all-1s, all-0s, mixed, and carry-in test vectors confirms matching sums and correct carry-out behaviour in every case, while gate-level delay analysis confirms the intended speed advantage over a ripple-carry design is restored.

Question 3: Optimizing Booth's Algorithm Multiplication

1. Identification of Errors / Bugs / Inefficiencies

- Redundant single-bit inspection: the design examines the multiplier one bit at a time (Q_n, Q_{n-1}) with a plain arithmetic shift right (ASR) of the combined AC/Q/Q-1 register set every cycle, even when consecutive bits are identical (00 or 11), which forces one shift-only cycle per bit regardless of pattern — this is standard Radix-2 Booth behaviour but is the direct source of excess cycles.
- Unnecessary add/subtract operations: strings of alternating 1s and 0s (e.g., 0101...) cause the algorithm to repeatedly ADD then SUBTRACT on consecutive cycles instead of recognizing the pattern and skipping redundant arithmetic, wasting cycles on operations that partially cancel out.

- No run-length grouping: the implementation does not group multiple bits together, so it cannot skip over long runs of identical bits (all 0s or all 1s) in a single step, unlike Radix-4/Booth-2 encodings which examine bit triplets.

2. Debugging Approach and Analysis

Tracing register contents (AC, Q, Q-1) cycle by cycle for a representative signed multiplicand and multiplier shows that for a multiplier bit pattern with long runs of 0s or 1s, or with frequently alternating bits, the design still executes one decision-and-shift cycle per bit, i.e., n cycles for an n -bit multiplier, with no cycles saved even when the Booth encoding recognizes a repeating pattern. This confirms the inefficiency lies in the granularity of bit inspection (Radix-2, one bit at a time) rather than in the correctness of the add/subtract decision table itself.

3. Corrected / Optimized Solution — Radix-4 (Modified) Booth's Algorithm

The multiplier is re-implemented using Radix-4 (Modified) Booth recoding, which examines overlapping bit triplets (Q_{i+1}, Q_i, Q_{i-1}) instead of a single bit pair, and performs one of the following actions per triplet, halving the number of cycles required:

Bit Triplet ($Q_{i+1} Q_i Q_{i-1}$)	Operation on AC	Shift
000	No operation	Shift right by 2
001	$AC = AC + M$	Shift right by 2
010	$AC = AC + M$	Shift right by 2
011	$AC = AC + 2M$	Shift right by 2
100	$AC = AC - 2M$	Shift right by 2
101	$AC = AC - M$	Shift right by 2
110	$AC = AC - M$	Shift right by 2
111	No operation	Shift right by 2

Because each step consumes two bits of the multiplier instead of one, the number of add/subtract-and-shift cycles is reduced from n to approximately $n/2$ for an n -bit operand, directly addressing the excess-cycle problem while preserving the exact numerical result guaranteed by Booth recoding.

4. Optimization Summary

- Adopt Radix-4 Booth recoding to halve the number of shift-and-decide cycles.
- Pre-compute and hold $2M$ (multiplicand shifted left by 1) in a register so the '+2M / -2M' cases require no extra cycle.
- Use arithmetic shift right by 2 positions per cycle, correctly sign-extending the MSB to preserve the sign of the partial product.
- Pipeline the add/subtract and shift stages where hardware area permits, further overlapping cycles for back-to-back multiplications.

5. Testing and Validation

The optimized Radix-4 Booth multiplier was tested against representative signed multiplicand/multiplier pairs, including operands with long runs of 1s, long runs of 0s, alternating bit patterns, and the most-negative representable value. In every case the final product matched the result obtained from direct signed multiplication and from the original Radix-2 Booth implementation, while the cycle count was reduced to approximately half — confirming the optimization improves speed without any loss of arithmetic accuracy.

Question 4: Debugging the Non-Restoring Division Algorithm

1. Identification of Errors / Bugs

- Incorrect quotient-bit assignment: the implementation sets the quotient bit to 1 only when the remainder register is strictly positive after the add/subtract step, instead of using the correct rule 'quotient bit = 1 if the sign of the remainder register (A) is 0 (non-negative), else 0.' This misclassifies a zero remainder.
- Wrong operand chosen on each step: the algorithm should subtract the divisor (M) when the previous remainder sign was non-negative and add M when the previous remainder sign was negative; the faulty version always subtracts M regardless of the previous sign, which is only correct for the very first step.
- Missing final correction step: after the last iteration, if the final remainder (A register) is negative, the algorithm must perform one final restoring addition of M to A to obtain the true remainder — this step is omitted, leaving a negative remainder in the result.

- Shift direction/alignment fault: the AQ register pair is not shifted left correctly before each add/subtract (the divisor is being aligned with the wrong bit position), causing partial remainders to be computed against a misaligned divisor.

2. Debugging Approach and Analysis

Manually tracing the non-restoring division of a representative dividend by a representative divisor, cycle by cycle, and recording the A (remainder), Q (quotient), and sign bit at every step isolates exactly which rule was violated:

- Step check 1: confirm A and Q are shifted left together by one bit before every add/subtract (this aligns the next dividend bit for use).
- Step check 2: confirm the correct operand is chosen — subtract M if the sign of A from the previous step was non-negative (0); add M if it was negative (1).
- Step check 3: confirm the new quotient bit Q₀ is set to the complement of the sign of the updated A (Q₀ = 1 if A ≥ 0, else Q₀ = 0).
- Step check 4: after n iterations, confirm that if the final A is negative, one restoring addition of M is performed to correct the remainder, and the corresponding quotient bit pattern is left unchanged.

Applying these checks to the faulty trace shows the divergence begins at the second iteration, where the algorithm subtracts M again instead of adding M after a negative remainder — this single rule violation propagates and corrupts every subsequent quotient bit.

3. Corrected Solution

- Initialize A = 0, Q = dividend, M = divisor, and run for n iterations (n = word length).
- At each iteration: shift the AQ register pair left by 1 bit.
- If the sign of A (from the previous step, or the initial 0 on the first step) is 0, set A = A - M; otherwise set A = A + M.
- Set the new Q₀ (LSB of Q) to 1 if the sign of the updated A is 0, else set it to 0.
- After the final iteration, if the sign of A is 1 (negative), perform one restoring step: A = A + M, to obtain the correct final remainder.

- The quotient is the final content of Q; the remainder is the final (corrected) content of A.

4. Optimization

Since non-restoring division already avoids the extra restoring add of the classical restoring-division algorithm on every cycle (it restores only once, at the end, if needed), the main further optimization is to pipeline the shift and add/subtract stages and to pre-decode the sign of A one cycle ahead so the add/subtract operand for the next cycle is selected without waiting for the full ALU result to settle, shortening the critical path per iteration. Using a carry-look-ahead or carry-save adder inside the ALU also reduces the per-cycle add/subtract latency.

5. Testing and Validation

Dividend	Divisor	Expected Quotient	Expected Remainder	Result After Fix
1011 (11)	0011 (3)	0011 (3)	0010 (2)	Q=3, R=2 (Correct)
1000 (8)	0010 (2)	0100 (4)	0000 (0)	Q=4, R=0 (Correct)
0111 (7)	0010 (2)	0011 (3)	0001 (1)	Q=3, R=1 (Correct)
1001 (9)	0100 (4)	0010 (2)	0001 (1)	Q=2, R=1 (Correct)

The corrected algorithm was traced against multiple dividend/divisor pairs, including cases that previously produced a negative uncorrected remainder. In each case the final restoring step yielded the correct non-negative remainder and the quotient matched direct integer division, confirming correctness after debugging.

Question 5: Optimizing IEEE 754 Double-Precision Floating-Point Computations

1. Identification of Performance Issues

- Over-precision for the task: double-precision (64-bit: 1 sign + 11 exponent + 52 mantissa bits) is used throughout the graphics pipeline even for operations such as colour blending, vertex transforms, and lighting calculations, where single-precision (32-bit) accuracy is visually and numerically sufficient.
- Frequent normalization and rounding overhead: every add/subtract/multiply operation on double-precision operands requires alignment of exponents, normalization of the mantissa, and rounding (per IEEE 754 round-to-nearest-even), each of which costs extra cycles compared to fixed-point or single-precision operations.
- Unnecessary type promotion: intermediate results are being implicitly promoted from single to double precision during mixed-precision expressions, forcing extra conversions and doubling memory bandwidth and register/cache usage without any corresponding accuracy benefit in the final rendered output.
- Repeated computation of invariant sub-expressions: some per-pixel or per-vertex floating-point calculations recompute constant or slowly-changing terms every cycle rather than caching them, adding avoidable floating-point operation counts.

2. Debugging Approach and Analysis

Profiling the arithmetic pipeline operation by operation shows that the bulk of the cycle cost is concentrated in double-precision multiply and divide operations used for routine transform and shading math, not in operations that genuinely require 52 bits of mantissa precision (such as accumulated physics simulation over very long run times). Comparing the dynamic range and precision actually required by each computation against what IEEE 754 double-precision provides confirms that most graphics operations are over-provisioned in precision relative to their accuracy requirements.

3. Optimization Techniques Recommended

- Precision downgrade where safe: replace double-precision (binary64) with single-precision (binary32) or half-precision (binary16) arithmetic for colour, texture, and per-pixel shading computations, reserving double precision only for accumulation-

sensitive stages (e.g., long-running physics integrators) where rounding error would otherwise accumulate.

- Fused multiply-add (FMA): replace separate multiply-then-add sequences with a single fused multiply-add instruction, which performs the multiplication and addition with a single rounding step, reducing both operation count and rounding error.
- Avoid unnecessary mixed-precision promotion: keep expressions in a single, consistently chosen precision so the hardware does not repeatedly convert between single and double precision.
- Constant folding / caching of invariant terms: precompute and cache sub-expressions that do not change between iterations (e.g., per-frame constants) instead of recomputing them per pixel or per vertex.
- Vectorized (SIMD) floating-point execution: batch independent single-precision operations together using SIMD instructions to exploit data-level parallelism, since the reduced 32-bit width also allows more operands to be processed per instruction than 64-bit double precision.

4. Expected Outcome / Trade-off Analysis

Moving routine graphics arithmetic from double to single precision approximately halves memory bandwidth and typically increases throughput on hardware where single-precision units are faster or more numerous than double-precision units, while the visual accuracy loss is negligible for rendering-scale computations. Using FMA reduces both the instruction count and the accumulated rounding error compared to separate multiply and add instructions. Precision-sensitive stages are deliberately left at double precision so overall numerical accuracy requirements for the application are still met.

5. Testing and Validation

Metric	Before Optimization (Double Precision)	After Optimization (Mixed/Single Precision + FMA)
Precision used	64-bit throughout	32-bit for shading/transform; 64-bit only where required
Arithmetic operations per pixel	Separate multiply + add	Fused multiply-add (FMA)

Metric	Before Optimization (Double Precision)	After Optimization (Mixed/Single Precision + FMA)
Memory bandwidth	Higher (8 bytes/value)	Reduced (4 bytes/value where applicable)
Rendering accuracy	Full double precision	Visually and numerically within tolerance

Benchmarking the pipeline before and after the change, with rendered output compared pixel-by-pixel against the double-precision baseline, confirms the optimized mixed-precision pipeline runs faster while remaining within acceptable numerical tolerance for the graphics application, validating that the optimization improves speed without a perceptible loss of required accuracy.